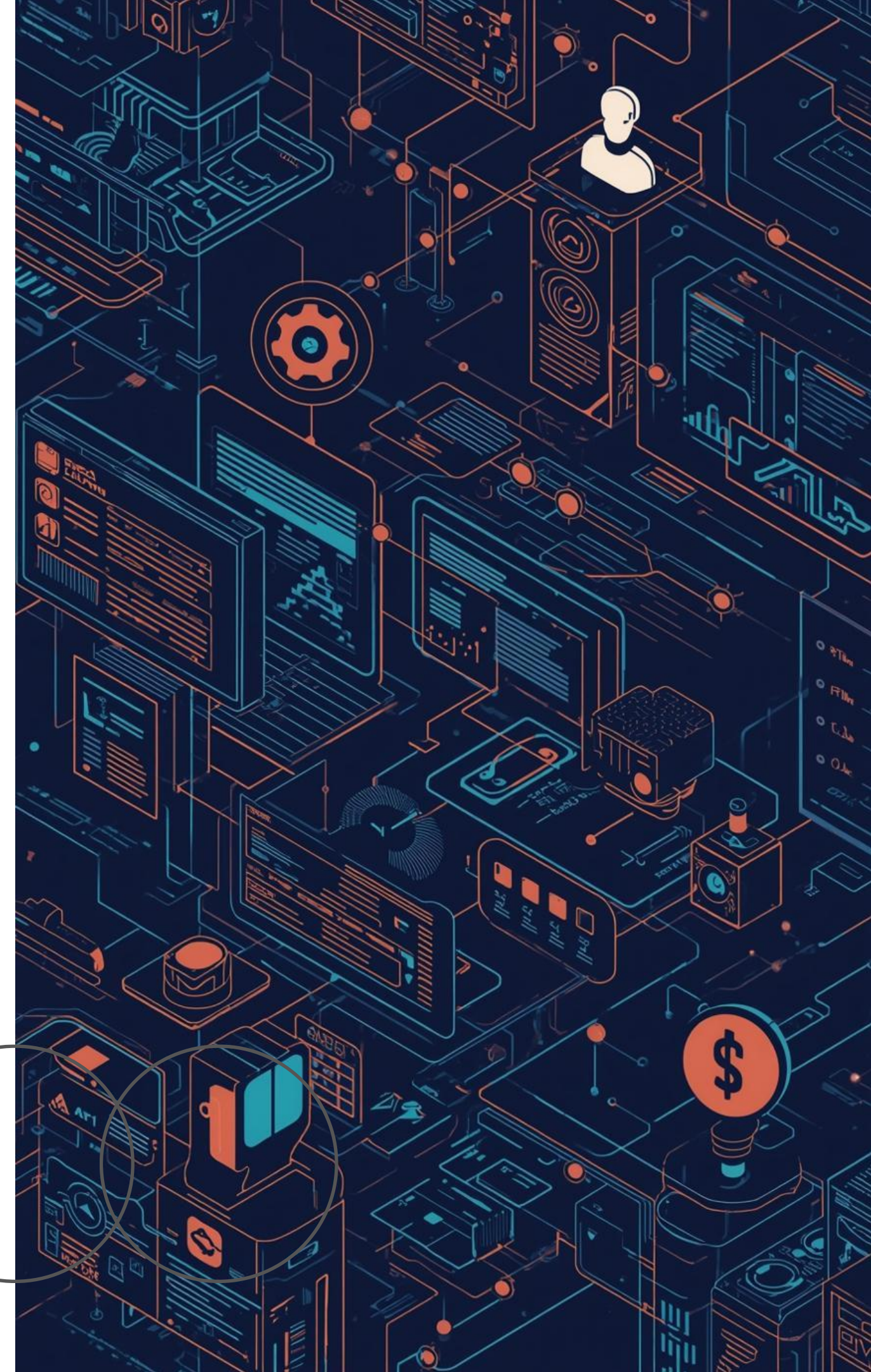
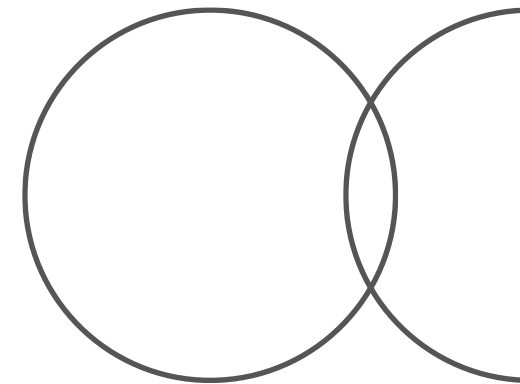


TMA SOLUTIONS

Agent Evaluation, *Optimization &* Safety

Presented by Tinh To



Eval agent is harder than made one

Most agents look better in demos than in production

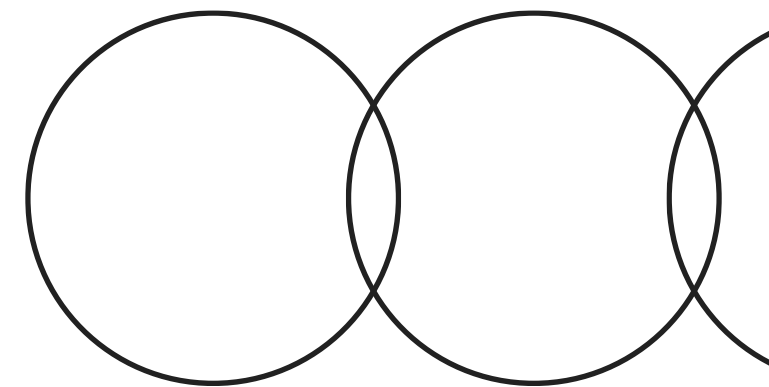
- Demos optimize for impressive answers.
- Happy cases

Many early agent prototypes look impressive in controlled environments

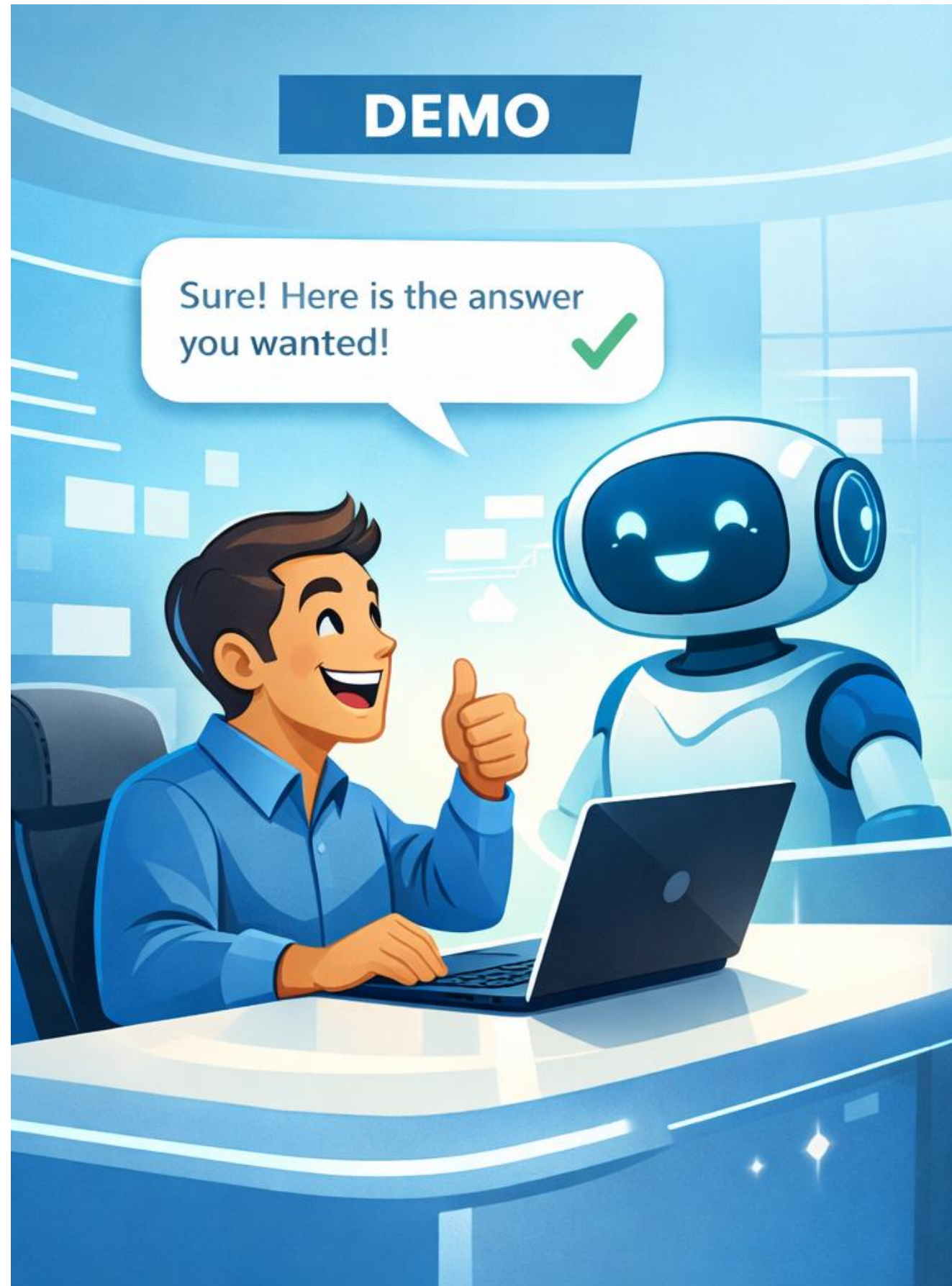
- But fail under production pressure.
- The most common reason is that teams optimize the visible answer instead of the system that produced it.

The most important factor

1. Accuracy
2. Hallucination
3. Performance
4. Toxicity



The META right now

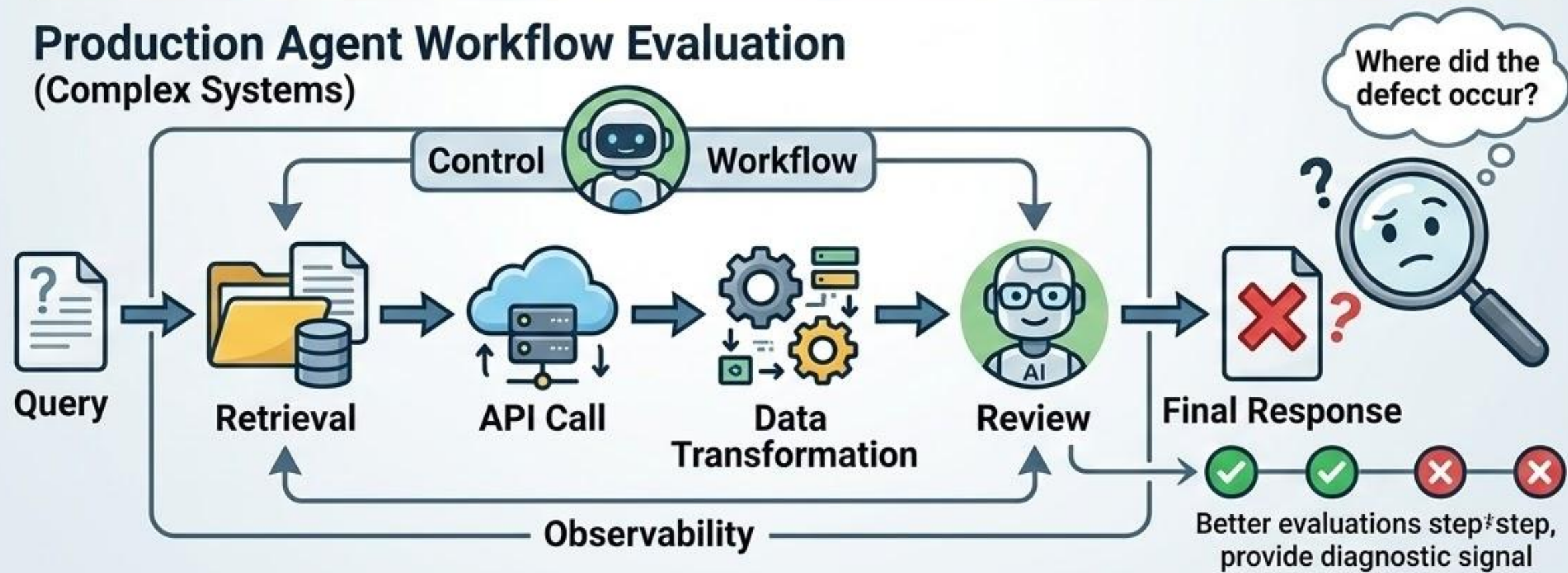


Part 1: Evaluations

Traditional Evaluation (Simple QA Benchmarks)



Production Agent Workflow Evaluation (Complex Systems)



Step 1: Data Collection (The Foundation)

You can't improve what you don't measure.

Capturing the right telemetry from your agent's executions is the mandatory first step.

- Traces (Macro): The full end-to-end journey (User prompt -> Agent routing -> DB query -> Final response).
- Observations/Spans (Micro): The individual, isolated steps within that trace.

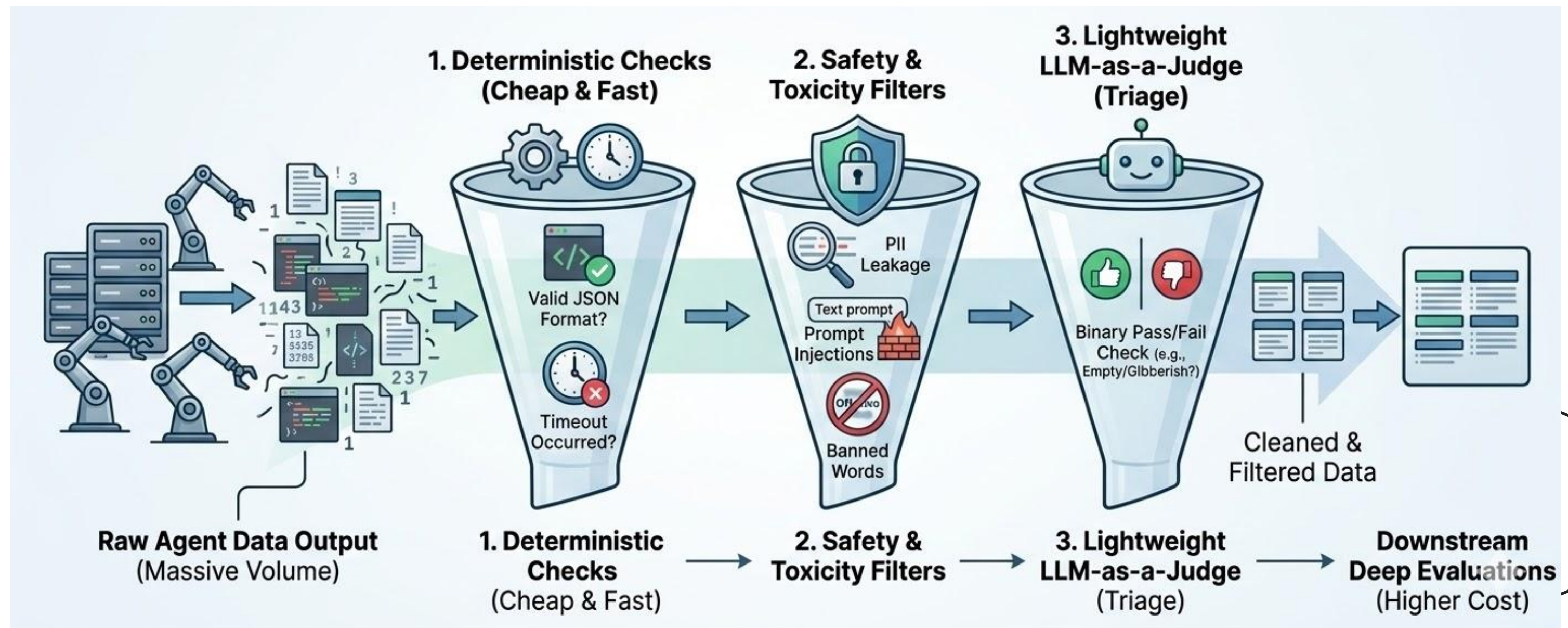
What to Collect First:

- Prioritize Performance metrics immediately: total execution time, time-to-first-token, and token costs.
 - Log the exact user I/O (Input/Output).
 - Track tool execution states (Success/Fail flags).
 - Takeaway: Start simple before trying to capture every internal agent thought process.
- 

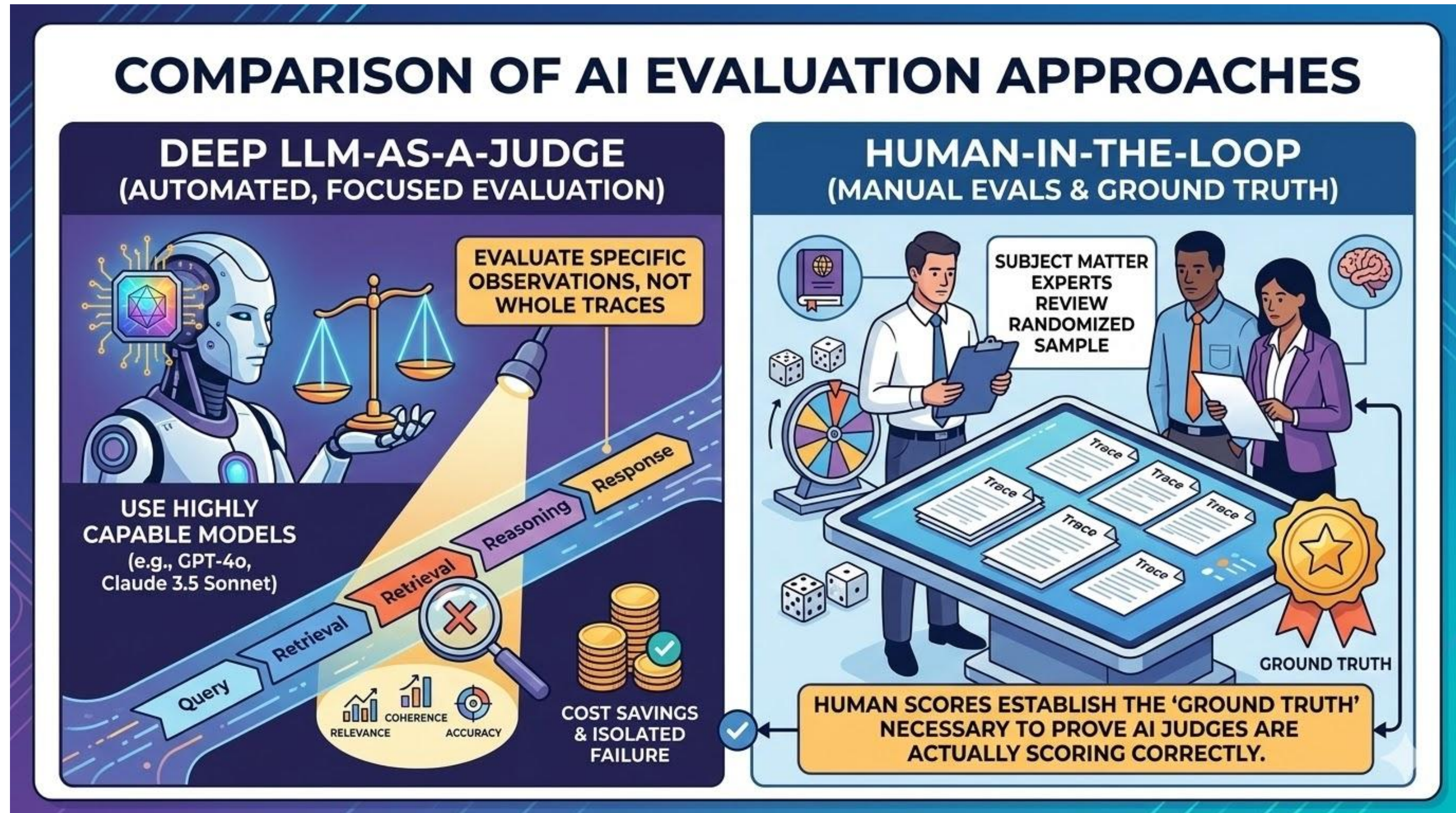
Step 2: The Automated Pipeline

The first line of defense and noise reduction.

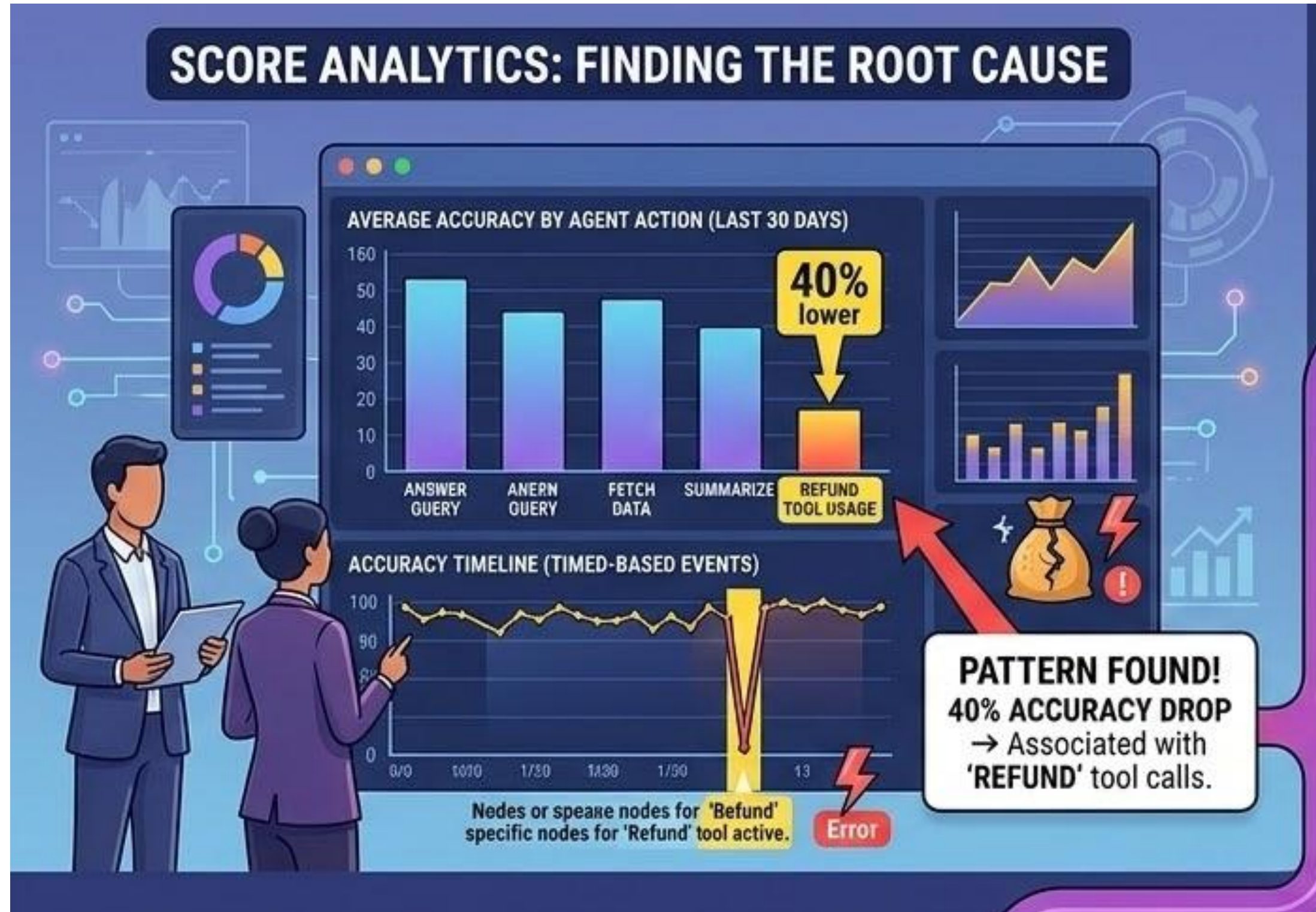
Agents generate massive amounts of data. You need a fast, automated layer to clean, format, and filter data before spending money on deep evaluations.



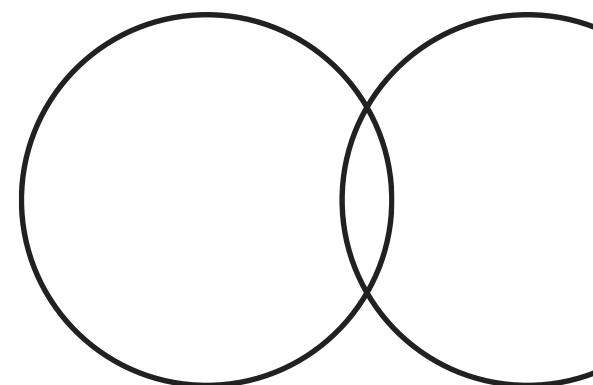
Step 3: The Evaluation Phase



Step 4: Turn metrics to meaningful insight



Part 2: The impossible pyramid



Tip 1- Model Selection & Swapping

Don't rebuild the car; just swap the engine.

The fastest and most impactful optimization is matching the right model family to the specific task. In a multi-agent system, not every node needs the same model.

The "Big 3" Model Profiles (Enterprise View):

- **1. The GPT Family (OpenAI):** The reliable generalist and deep thinker. Industry-leading function calling and complex planning (o-series). Great for routing and management.
- **2. The Claude Family (Anthropic):** The meticulous coder and speed-reader. Exceptional at coding, natural tone, and strictly adhering to complex system prompts within massive context windows. Great for RAG synthesis and code generation.
- **3. The Gemini Family (Google):** The massive multimodal data engine. Unprecedented context windows and native multimodal capabilities (audio/video/text). Great for bulk processing and filtering massive datasets.

Tip 2 - Context Matters

Context overload:

- Ignoring instructions in the middle of a prompt
- Conflicting personas
- Not enough context window

Hallucination:

- Produce wrong and dangerous result
- Unreliable tool call, tool looping

Performance issue:

- Took too long to run
- Burn token at alarm rate



Tip 2 - Manage context window

Smart vs. Dumb Zone Management

- The "Smart" Zones (Top & Bottom): Place your critical system instructions, persona definitions, and final strict output constraints here.
- The "Dumb" Zone (Middle): Place the passive, bulky payload (RAG context, long documents) in the middle.

Knowledge Tiering: Pre-baked vs. Dynamic

- Pre-Bake (Internal Knowledge): Core routing logic, output schemas, and high-value "few-shot" examples go directly into the system prompt.
- Dynamic (External Knowledge): Orchestrate tools to fetch the rest (RAG for static docs, Search for real-time facts).

Token Optimization (TOON Format)

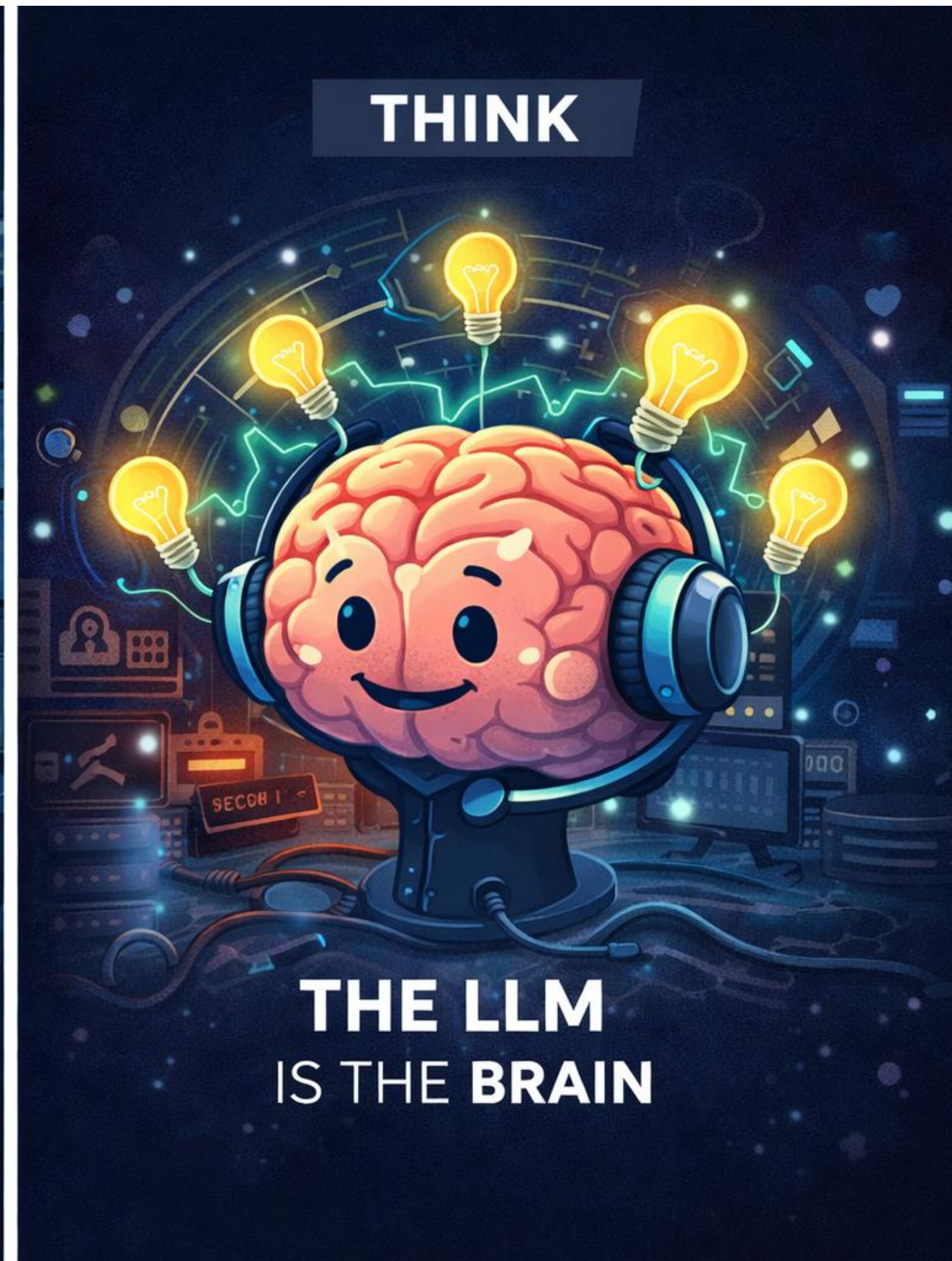
- Standard JSON is heavy on tokens. Translate intermediate data payloads into TOON (Token Optimized Object Notation) or YAML to drop syntactic noise, reducing your prompt's token footprint by 20% to 40% per call.

Tip 3 - Divide and conquers

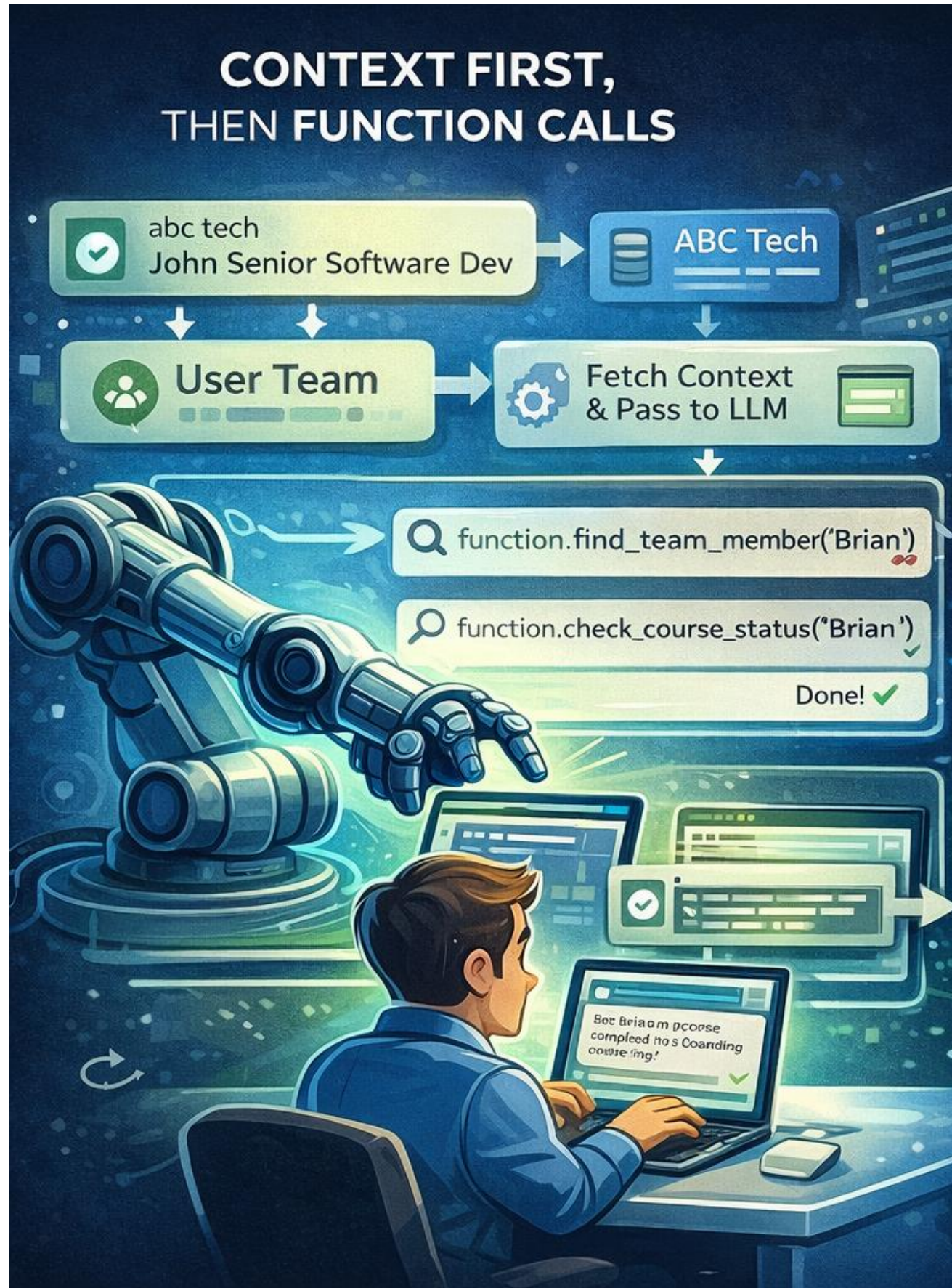
Solving a Customer's Problem



Tip 3 - Software Act, AI Think



Tip 3 - Software Act, AI Think



Tip 4 - Multi-agent

Squeezing the last 20% of performance out of the node.

- **Few-Shot Prompting (Show, Don't Tell):** Provide 3 to 5 perfect examples in the system prompt. It drastically reduces hallucination and often allows you to downgrade to a cheaper model.
- **Cross-Model Validation (The Second Opinion):** Pair a smart model with a cheap model. Have Claude write a financial report, and have Gemini Flash extract the numbers to verify they match the database.
- **Cross-Agent Handoffs (Passing the Baton):** Agents shouldn't just talk; they need to transfer state. Use tools to package summarized context and blockers so the receiving agent doesn't have to re-read the entire raw chat log.

Tip 4 - Multi-agent

The Behavior Contract as a Positive Patient Experience

By Ramiro Valdez, PhD - ESRD Network Patient Services Director

Behavior contracts are being used more and more to change undesirable, problematic patient behavior. While behavior contracts can be very effective tools for modifying self-destructive or aggressive behaviors, a poorly written or implemented behavior contract may lead to a patient's refusal to cooperate and/or continued or increased problematic behavior. Appropriate use of a behavior contract can, however, be a positive learning experience leading to mutual understanding and improved relations between patients and staff. In fact, a well written, mutually agreed upon contract could be the singular most effective way to help a patient change behavior.

Appropriate and effective behavior contracts set a goal of a change in problematic behavior. Problematic behavior is any behavior that is self-injurious or creates a hostile environment in the clinic and can include such behaviors as signing off dialysis early and skipping treatments.

An effective behavior contract incorporates all parties working together to achieve mutual goals. In the dialysis setting, the parties are the patient, the physician and the clinic staff. **Each has a role to play in order to achieve the goal.** Essential components of a behavior contract include statements of responsibility from the patient and the clinic, the use of positive behavioral terms, statements of responsibility from the patient and the clinic, utilization of a staff monitor, a specified time frame, and a follow-up review.

Behavioral Terms

- The goal is stated as the desired behavior, **NO** as the behavior to be stopped. Asking for desired behaviors is more effective than listing undesirable behaviors. For example, "will show up for every treatment" is better than "will not skip treatments".
- The desired change in the patient's behavior is stated in POSITIVE, OBSERVABLE behaviors. Examples of positive phrases include "will speak in a normal voice" or "will address complaints in the nurse's room instead of other patients." Care is taken not to insult the patient.
- Several items may be included in the contract, but the patient may feel overwhelmed if too many behavior changes are included. The goal is to modify behavior, not make this person the best patient in the clinic.
- The physician, administrator, social worker and nurse manager should meet with the patient in private to discuss mutual responsibilities and develop the behavior contract. Include a member of the patient's family if requested by the patient.
- All parties involved in fulfilling terms of the contract should sign the contract.
- The behavior contract should **never** be written without patient input and then forced on the patient.
- Staff/patient or staff-staff conversations about the behavior contract are confidential and should be held in private (i.e., not in front of other patients).

Clinic Responsibility

- A contract is a statement of responsibility or tasks that **both** parties will perform in order to achieve a goal. A behavior contract points out not only the role of the patient, but the role of the clinic in achieving desired goals.
- The patient is given an opportunity to point out areas where the clinic needs to make some changes. The areas that most patients suggest are greater patient sensitivity, professional behavior and cross-cultural communication.
- If patients are not specifically asked to make suggestions, they will probably not speak up. They don't know they have this right. Staff should inform patients they **have** the right to make suggestions for improving the clinic.

- ➔ A behavior contract that calls for the patient to make changes to behavior while not requiring action from the clinic, places the patient in a losing situation.
- ➔ It is largely because of this one omission that behavior contracts rarely work.



Microsoft Excel - MSC-LIMS Sample Logbook

Log LIMS Samples

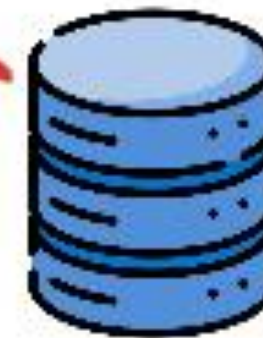
This workbook demonstrates how MSC-LIMS samples can be logged using sample table at left (named range LIMSInterface) identifies the cells where sample data is obtained from a single spreadsheet row. The LIMS Field names in the table must names. The worksheet with samples to log must be the first worksheet in the workbook identified in the LIMSInterface table to the left, must point to cells on the first work

The minimum fields required to log an MSC-LIMS sample are project, location, sa date. Make sure the cells for these fields have valid data. Note that the LIMS fee ProjectID, LocationID, etc.) identify the MSC-LIMS internal ID value, which can be query" on the SQL tab of the MSC-LIMS query controls after querying for the site Projects, Locations, Sample Types, and Samples reports to the empty MSC-LIMS internal ID.

To log MSC-LIMS samples from this workbook, open MSC-LIMS, use Batch Login empty batch such as a "Dynamic Batch" then return to this workbook. Use the [J] add the workbook's samples to the Batch Login screen. Return to MSC-LIMS to

For more information contact:

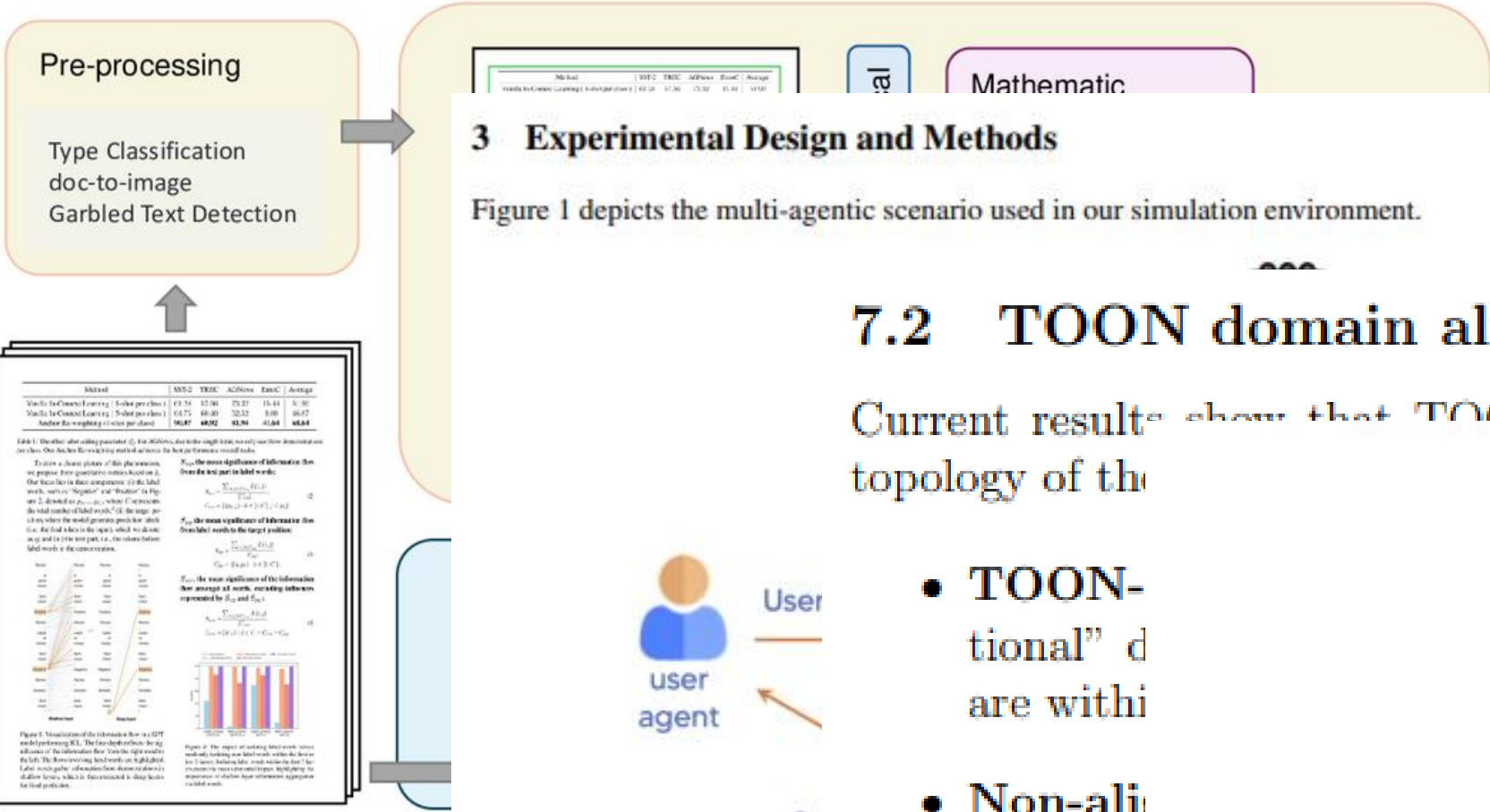
LIMS Field	Range
ProjectID	A10:A18
LocationID	L1:L5
SampleTypeID	A9:M9
SampleID	N0:N5
CustomerID	O0:O5
CollectionDate	A9:A18
CollectionTime	B9:B18
CustomerSampleID	C9:C18
Description	D9:D18
Notes	



Tip 4 - Multi-agent



Tip 5 - Research



7.2 TOON domain alignment and application:

Current results show that TOON's performance while significant is strictly dictated by the topology of the



- TOON-tional" d are withi
- Non-ali or non-u

Figure

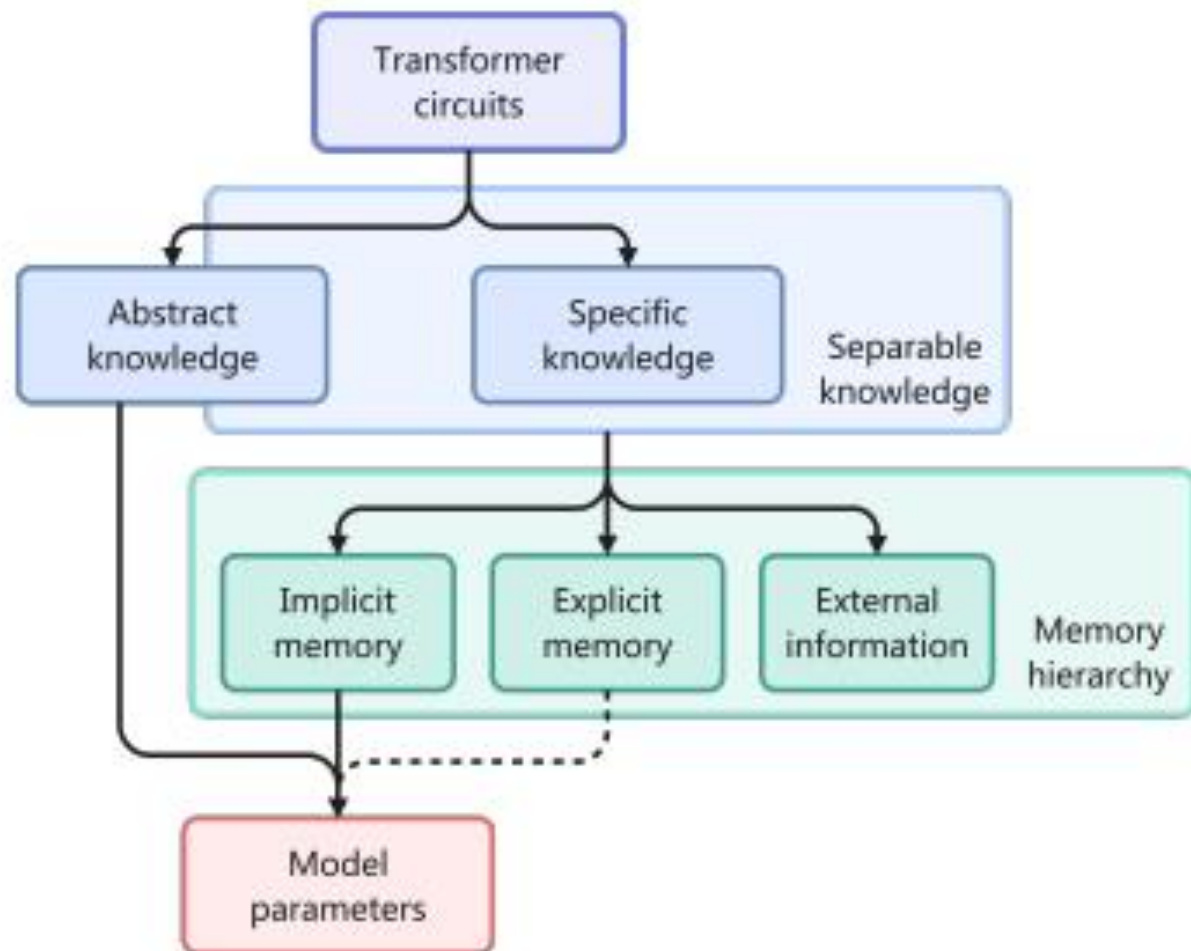


Figure 2 Categorization of LLM knowledge, including the memory hierarchy. The explicit memories, extracted from model activations, lie half-way between raw data and model parameters, so a dotted line is used to indicate that they may or may not be regarded as parameters. Reproduced from [1].

Part 3 - Safety



Paradox 1 — The Smart Agent vs. The Dumb Agent

Smart Agent (Frontier Model)



- 🧠 understands intent → better at detecting prompt injection
- 💡 more robust reasoning
- 🧩 adapts to complex edge cases

⚠️ Risk:



- 🧠 high capability → high blast radius
- 🧠 can generate harmful actions if misaligned
- 🧠 harder to fully constrain

Dumb Agent (Constrained Model)



- 🧠 narrow capability → limited damage
- 🧠 predictable behavior
- 📄 easier to sandbox and verify

⚠️ Risk:



- 🧠 easier to manipulate
- 🧠 lacks contextual awareness
- 🧠 cannot detect subtle malicious intent



Insight

Intelligence improves judgment. Constraints limit damage.
You need both — but they conflict.

Part 3 - Safety



Paradox 2 – The Simple Chain vs. The Multi-Agent Swarm

Swarm (Multi-Agent System)



- 🧠 internal checks (Dev vs Reviewer vs Security)
- 🧠 self-correction emerges
- 🧠 higher problem-solving capability

⚠️ Risk:

- 🧠 emergent behavior
- 🧠 cascading failures
- 🧠 difficult to audit
- 🧠 exponential token/cost growth

Simple Chain



- 🧠 deterministic flow
- 🧠 easy to trace and debug
- 🧠 strong compliance and auditability

⚠️ Risk:

- 🧠 no internal validation
- 🧠 brittle to edge cases
- 🧠 limited intelligence

VS



Insight

More agents ≠ more safety. More agents = more surface area.

Part 3 - Safety

1. Permission Boundary

What the agent is allowed to do

- read vs write access, API scopes, database operations

Enforce by execution layer (not prompt)

2. Blast Radius

Maximum damage if the agent fails

- can it delete data? send emails? execute code? trigger payments?

Never give an agent more power than the worst-case failure can tolerate

3. Knowledge Boundary

What the agent is allowed to know

- internal documents, PII / sensitive data, system schemas, private APIs

Risk: data leakage, prompt injection via hidden data

4. Action Surface

What actions the agent can attempt

- tool set size, number of integrations, allowed operations

More tools = more attack surface

5. Observability Boundary

What you can see and audit

- step-level logs, tool calls, reasoning traces, cost tracking

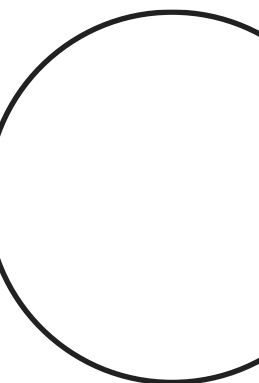
If you can't observe it → you can't trust it

6. Cost Boundary

How much the system is allowed to spend

- max tokens per trace, max iterations, max parallel calls

Prevents runaway loops



Part 3 - Safety

1. Smart at the Edge (Router Layer)

- detect intent
- classify risk
- route tasks
- reject malicious prompts early

☞ high intelligence, low authority

2. Dumb in the Core (Worker Layer)

- execute narrow tasks
- limited scope
- minimal permissions

☞ low intelligence, low blast radius

3. Hard Guardrails (Execution Layer)

- enforce permissions
- deny unsafe actions
- sanitize outputs

☞ deterministic control

4. Circuit Breakers

- max iterations
- max cost
- timeout
- fail-safe stops

5. Observability + Audit

- full trace logging
- replay capability
- anomaly detection

The Iron Law (very strong concept)

🔒 **The Iron Law of Agent Design:**

Capability without constraint becomes liability or sharper:

Every increase in intelligence must be matched with an equal increase in control